

# Implementation Self-Assessment Test Report

## Key Encapsulation-DKEM-x86-v1.0 (ICCS 2026-06 electronic submission)

*Per the ICCS x86-architecture Implementation Self-Assessment Guideline (2026-06, CN & EN editions)*

This report records, for DKEM (Key Encapsulation (KEM)) on the x86 architecture, the basic information, test environment, functional correctness, performance, resource consumption and transfer/storage sizes, plus a raw-evidence index. Verifiable technical evidence only; no security claim. Object: the will-submit electronic package. Test date: 2026-06-24.

## 1 Basic Information

- **Algorithm:** DKEM (PKCKEM-380333)
- **Category:** Key Encapsulation (KEM)
- **Implementation version:** v1.0 (ICCS 2026-06 electronic submission)
- **Architecture:** x86
- **Implementations:** Reference (portable ISO C99) and Optimized (x86-64 AVX2)
- **Instances & security level:** DKEM-128/-256/-512 — 128/256/512-bit classical security level

Table 1 Parameter sets

| Instance | N   | Q    | K | Security |
|----------|-----|------|---|----------|
| DKEM-128 | 256 | 3329 | 2 | 128 -bit |
| DKEM-256 | 256 | 3329 | 4 | 256 -bit |
| DKEM-512 | 512 | 7681 | 4 | 512 -bit |

## 2 Test Environment

- **Hardware:** Intel Core i9-10885H @2.40 GHz (single-core measurement), 64 GiB RAM
- **OS:** Ubuntu 24.04 (WSL2, kernel 6.x)
- **Compiler / build:** GNU gcc 13.3.0; bash build.sh
- **Guideline baseline:** native Linux, gcc  $\geq 8.5.0$ , kernel  $\geq 4.19$  — met
- **Memory footprint:** process peak resident memory  $\approx 1\,876$  KB (see §6); static image size in §6
- **Measurement window (UTC):** 2026-06-23T20:56:28Z  $\rightarrow$  2026-06-23T20:59:36Z

## 3 Versions and Compile Configuration (Registration)

### 3.1 Reference (ISO C99, no SIMD)

```
gcc -std=c99 -Wpedantic -Wall -Wextra -O2  
-fcommon -DDKE_FORCE_SCALAR -DDKE_MODE=<128|256|512> -DDKE_HASH=0 -DDKE_RANDOM=0
```

### 3.2 Optimized (x86-64 AVX2)

```
gcc -O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer  
-std=c99 -Wpedantic -Wall -Wextra -fcommon -DDKE_HASH=0 -DDKE_RANDOM=0
```

- **Target ISA:** x86-64 + AVX2
- **RNG:** deterministic DRNG from API\_PKC (DKE\_RANDOM=0); KAT seeds from DRNG; reproducible.
- **Hash / XOF:** SM3 only via the official auxfunc (sm3hash/pseudoXOF/pseudohash), DKE\_HASH=0; no alternative hash built.
- **Third-party:** the x86-64 AVX2 optimization reuses several low-level building blocks from pq-crystals / PQClean (AVX2 constants / NTT / polynomial / rejection sampling / reduce; public domain, Apache-2.0, MIT) - external third-party, outside the B.3 ownership scope; the hand-written NTT assembly under avx2-linux/ is original work by proposal-team member Liu Rui (within B.3). The portable Reference implementation is plain ISO C99 with no third-party code. See Implementations/LICENSES/.
- **Protected files:** drng.c/.h, auxfunc.c/.h, KAT\_\*.c byte-for-byte unchanged.

## 4 Functional Test (KAT)

x86-64 (gcc): Reference + Optimized, 6 builds over 3 instances, all KAT outputs byte-identical to Test\_Vectors — 6/6 PASS (Ref  $\equiv$  Opt).

## 5 Performance Test (average time + average cycles)

Averages (DKE\_RANDOM=0, seeds from the deterministic DRNG). N=300 for KEM, N=128 for KEX (both  $\geq 100$ , per guideline §3.3). Each iteration is timed with clock\_gettime(CLOCK\_MONOTONIC) and CPU cycles via RDTSC; single core throughout.

Table 2 Average time ( $\mu$ s, keygen / encaps / decaps)

| Instance | Reference                | Optimized                |
|----------|--------------------------|--------------------------|
| DKEM-128 | 66.72 / 80.21 / 94.73    | 40.21 / 43.31 / 45.00    |
| DKEM-256 | 212.80 / 211.79 / 229.81 | 131.72 / 132.42 / 135.09 |
| DKEM-512 | 648.52 / 689.76 / 741.85 | 549.65 / 565.40 / 569.88 |

Table 3 Average CPU cycles (RDTSC, keygen / encaps / decaps)

| Instance | Reference                   | Optimized                   |
|----------|-----------------------------|-----------------------------|
| DKEM-128 | 159 721 / 192 088 / 226 929 | 105 171 / 113 311 / 117 731 |
| DKEM-256 | 509 765 / 507 604 / 550 777 | 329 137 / 330 887 / 337 571 |

| Instance | Reference                         | Optimized                         |
|----------|-----------------------------------|-----------------------------------|
| DKEM-512 | 1 555 048 / 1 654 277 / 1 778 708 | 1 317 471 / 1 355 246 / 1 365 954 |

The Optimized speedup is concentrated in the NTT / polynomial arithmetic (hand assembly). The hash is SM3 via the official auxfunc (scalar C, a compliance requirement), so the end-to-end speedup is moderate.

## 6 Resource Consumption (static image + peak memory)

Static image size from GNU size (text+data+bss, bytes) — storage footprint of code + constants + static data; peak resident memory from getrusage(RUSAGE\_SELF).ru\_maxrss (KB) — runtime memory footprint. Single core, single process.

Table Static image and peak memory

| Instance | Ref static | Opt static | Peak RSS |
|----------|------------|------------|----------|
| DKEM-128 | 35 757 B   | 130 849 B  | 1 876 KB |
| DKEM-256 | 36 957 B   | 133 043 B  | 1 876 KB |
| DKEM-512 | 38 461 B   | 123 167 B  | 1 876 KB |

Note: the Optimized image is markedly larger than Reference due to inlined hand assembly and -flt0; peak RSS is essentially flat across instances ( $\approx 1.8$  MB), dominated by the process baseline and on-stack key material.

## 7 Transfer and Storage Sizes (bytes)

Table DKEM key / ciphertext / shared-secret sizes

| Instance | Public key | Secret key | Ciphertext | Shared secret |
|----------|------------|------------|------------|---------------|
| DKEM-128 | 800        | 1 600      | 800        | 32            |
| DKEM-256 | 1 568      | 3 136      | 1 600      | 32            |
| DKEM-512 | 3 392      | 6 784      | 3 136      | 64            |

## 8 Conclusion

All three DKEM instances are functionally correct on the x86 architecture (KAT byte-identical to Test\_Vectors, Ref  $\equiv$  Optimized) and build reproducibly from the submitted automated scripts. Average time, average cycles, static image and peak memory, target ISA, compile parameters, RNG method and hash backend are all registered above.

## 9 Raw Evidence Index

The following raw data and scripts ship with the self-assessment materials package and allow independent reproduction of every result in this report:

- **3-Assessment-Code-and-Data/Raw-Evidence/assess\_x86.txt** — raw performance/memory

output for this architecture (with start/end timestamps)

- **3-Assessment-Code-and-Data/run\_assess.sh** — unified performance/memory/KAT harness (with embedded bench sources)
- **2-Algorithm-Source-Code/Implementations/.../build.sh** — per-instance deterministic build scripts (source of functional KAT)
- **3-Assessment-Code-and-Data/Test\_Vectors/** — reference vectors (byte-for-byte KAT comparison targets)

All timestamps are UTC; measurement host, compiler and window are in §2.